UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA
KATEDRA ZA AUTOMATIKU I UPRAVLJANJE SISTEMIMA

Identifikacija sistema
Parametarska identifikacija

(deo C)

Identifikacije parametara sistema
čiji je model nelinearan po nepoznatim parametrima

Modeliranje i simulacija sistema

Iterativne metode parametarske identifikacije

- Posmatra se vremenski diskretan model sistema:
  $$\boldsymbol{x}(\mathrm{k}+1) = f(\boldsymbol{x}(k), u(k), \boldsymbol{q}), \qquad \boldsymbol{x}(0) = \boldsymbol{x}_0$$
  $$y(k) = \varphi(\boldsymbol{x}(k), u(k), \boldsymbol{q})$$

  gde je izlaz modela nelinearan po parametrima. Radi jednostavnosti model ima jedan ulaz i jedan izlaz
- Određivanje parametara prema kriterijumu opt.:
  $$\mathcal{J}(\boldsymbol{q}) = \frac{1}{2}\sum_{k=0}^{K-1} e_k^2(\boldsymbol{q})$$

  $$e_k(\boldsymbol{q}) = e(k, \boldsymbol{q}) = y_v(k) - y(k, \boldsymbol{q}),$$
  $$k = 0,1, \dots, K-1$$

  tj. traženje $\widehat{\boldsymbol{q}}$ za koje je $\mathcal{J}$ minimalno: $\min\limits_{\boldsymbol{q}} \mathcal{J}(\boldsymbol{q})$

- Rešenje $\min \mathcal{J}$ se dobija iterativno
  $$\boldsymbol{q}_{n+1} = \boldsymbol{q}_n + \Delta\boldsymbol{q}_n$$

  gde je n oznaka iteracije, a ne merenja!

- Početna procena parametara je $\boldsymbol{q}_0$
- Kriterijum zaustavljenje algoritma:
  $$\|\Delta\boldsymbol{q}_n\| = \sum_{i=0}^{r} \Delta q_i^2 < \varepsilon_q$$

  ili relativan odnos
  $$\left\|\frac{\Delta\boldsymbol{q}_n}{\boldsymbol{q}_n}\right\| < \varepsilon_q$$

  ili
  $$\mathcal{J}(\boldsymbol{q}_n) < \varepsilon_J$$

  Dodatan uslov: $n \leq n_{max}$

Algoritmi (ponavljanje)

- Gradijentni algoritam

$$\Delta \boldsymbol{q}_n = -h\nabla \mathcal{J}(\boldsymbol{q}_n), \qquad h > 0$$

- Gaus-Njutn algoritam

$$\Delta \boldsymbol{q}_n = -\left(\boldsymbol{S}^T(\boldsymbol{q}_n)\boldsymbol{S}(\boldsymbol{q}_n)\right)^{-1}\nabla \mathcal{J}(\boldsymbol{q}_n)$$

- Lavenberg-Markart algoritam

$$\Delta \boldsymbol{q}_n = -(\boldsymbol{S}^T(\boldsymbol{q}_n)\boldsymbol{S}(\boldsymbol{q}_n) + \lambda_n \boldsymbol{I})^{-1}\nabla \mathcal{J}(\boldsymbol{q}_n)$$

gde se λ_n menja

$$\frac{\partial \mathcal{J}}{\partial q_i} = -\sum_{k=0}^{K-1} e(k,\boldsymbol{q})\frac{\partial y(k,\boldsymbol{q})}{\partial q_i}, \qquad i = 1,2,\dots,r$$

$$\nabla \mathcal{J}(\boldsymbol{q}) = \frac{\partial \mathcal{J}}{\partial \boldsymbol{q}} = \begin{bmatrix} \frac{\partial \mathcal{J}}{\partial q_1} \\ \frac{\partial \mathcal{J}}{\partial q_2} \\ \dots \\ \frac{\partial \mathcal{J}}{\partial q_r} \end{bmatrix} = -\boldsymbol{S}^T(\boldsymbol{q})\,\boldsymbol{E}(\boldsymbol{q})$$

$$\boldsymbol{U}(k,\boldsymbol{q}) = \begin{bmatrix} \frac{\partial y(k,\boldsymbol{q})}{\partial q_1} & \frac{\partial y(k,\boldsymbol{q})}{\partial q_2} & \dots & \frac{\partial y(k,\boldsymbol{q})}{\partial q_r} \end{bmatrix}^T$$

$$\boldsymbol{S}(\boldsymbol{q}) = \begin{bmatrix} \boldsymbol{U}^T(0,\boldsymbol{q}) \\ \boldsymbol{U}^T(1,\boldsymbol{q}) \\ \dots \\ \boldsymbol{U}^T(K-1,\boldsymbol{q}) \end{bmatrix}, \boldsymbol{E}(\boldsymbol{q}) = \begin{bmatrix} e(0,\boldsymbol{q}) \\ e(1,\boldsymbol{q}) \\ \dots \\ e(K-1,\boldsymbol{q}) \end{bmatrix}$$

Poređenje primene algoritama

Rešavanje sistema nelinearnih algebarskih jednačina $f_k(\boldsymbol{x}) = 0$

- Ukupna greška po svim jednačinama

$$\mathcal{J}(\boldsymbol{x}) = \frac{1}{2}\sum_{k=1}^{m} e_k^2(\boldsymbol{x})$$

- greška

$$e_k(\boldsymbol{x}) = f_k(\boldsymbol{x})$$

- Za rešavanje se obezbeđuje
  - Vektor funkcija $f_k(\boldsymbol{x}), k = 1,2, \ldots m$
  - Jakobijan, tj. izvodi svih $\boldsymbol{f}$ po svim $\boldsymbol{x}$

Parametarska identifikacija nelinearnih modela

- Ukupna greška po svim merenjima

$$\mathcal{J}(\boldsymbol{q}) = \frac{1}{2}\sum_{k=0}^{K-1} e_k^2(\boldsymbol{q})$$

- greška

$$e_k(\boldsymbol{q}) = y_v(k) - y(k, \boldsymbol{q})$$

- Za rešavanje se obezbeđuje
  - Funkcija izlaza modela $y(k, \boldsymbol{q})$ i merenje izlaza $y_v(k)$
  - Funkcije osetljivosti, tj. izvodi izlaza po svim parametrima $\boldsymbol{q}$

Gaus-Njutnov algoritam - Implementacija

```
function identgn(fun, y, u, q0;
                epsq=1e-6, epsJ=1e-6, kmax=100)
    # Parametarska identifikacija Gaus-Njutnovim alg.
    k = 0
    q = q0
    Δq = J = Inf

    while norm(Δq) > epsq && J > epsJ && k < kmax
        (fy, S) = fun(q, u)
        S = S'
        e = y .- fy
        grad = -S' * e
        Δq = -(S' * S) \ grad   # !!!
        q += Δq
        J = 0.5 * e' * e
        k += 1
    end

    sd = sqrt(2*J/length(u))
    return (q, J, sd, k)
end
```

`fun` treba da vrati:

- vrednost izlaza po funkciji modela $y(k, \boldsymbol{q})$ i
- vektor vrednosti funkcija osetljivosti $\frac{\partial y(k,\boldsymbol{q})}{\partial q_i}, i = 1,2, \dots r$

Radi poređenja, implementacija Gaus-Njutn algoritma za rešavanje sistema nelinearnih algebarskih jednačina.

```
function GausNjutn(fun, x0; epsx=1e-6,
                   epsJ=1e-6, kmax=100, logs=false)
    k = 0               # broj iteracija
    x = x0
    Δx = J = Inf
    stat = []
    while norm(Δx) > epsx && J > epsJ && k <= kmax
        (f, S) = fun(x)
        Δx = -(S'*S) \ (S'*f)
        x += Δx
        J = 0.5 * f' * f
        k += 1
        if logs
            push!(stat, x)
        end
    end
    return (x, J, k, stat)
end
```

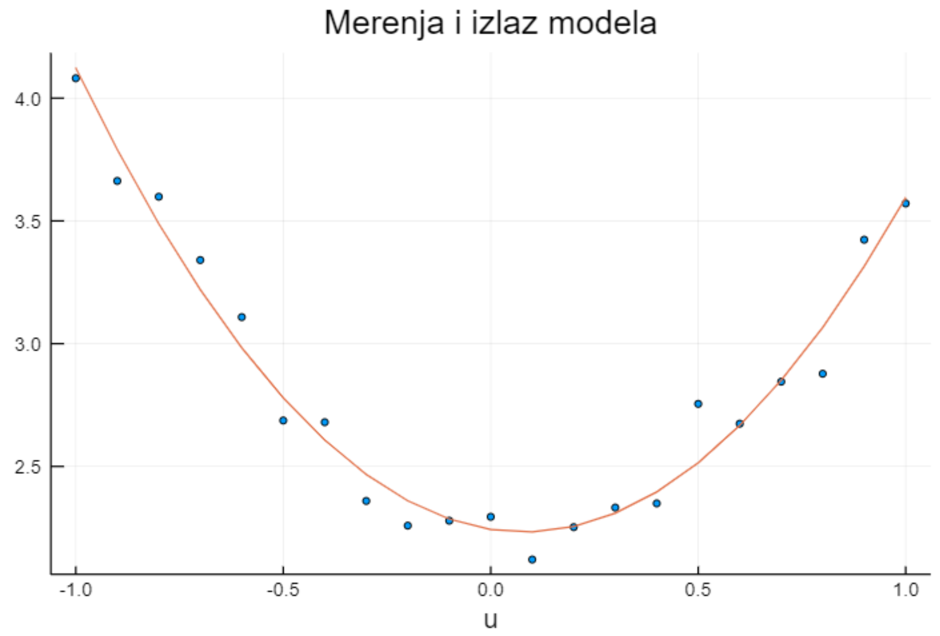
Gradijentni algoritam - Implementacija

```
function identgrad(fun, y, u, q0, h;
                epsq=1e-6, epsJ=1e-6, kmax=100)
    # Parametarska identifikacija gradijentnim algortimom
    k = 0              # broj iteracija
    q = q0
    Δq = J = Inf

    while norm(Δq) > epsq && J > epsJ && k < kmax
        (fy, S) = fun(q, u)
        S = S'
        e = y .- fy
        grad = -S' * e
        Δq = -h * grad      # !!!
        q += Δq
        J = 0.5 * e' * e
        k += 1
    end

    sd = sqrt(2*J/length(u))
    return (q, J, sd, k)
end
```

Levenberg-Markart algoritam - Implementacija

```
function identmarq(fun, y, u, q0; h=0.5, epsq=1e-6, epsJ=1e-6, kmax=100)
    k = 0                   # broj iteracija
    q = q0
    Δq = Js = Inf           # prethodna vrednost J je Js
    λ = 1/h; ni = 1.2
    while norm(Δq) > epsq && Js > epsJ && k < kmax
        (fy, S) = fun(q, u)
        S = S'
        e = y .- fy
        J = 0.5 * e' * e
        grad = -S' * e
        Δq = -((S'*S) + λ*I) \ grad
        k += 1
        if J > Js
            λ = λ * ni
        else
            λ = λ / ni
            q += Δq
        end
        Js = J
    end
    sd = sqrt(2*Js/length(u))
    return (q, Js, sd, k)
end
```

Primer 1



Identifikovati parametre parabole na osnovu datih merenja

Rešenje:
$y(\boldsymbol{q}, u) = q_1 u^2 + q_2 u + q_3$

$$\boldsymbol{U}(k, \boldsymbol{q}) = \begin{bmatrix} \dfrac{\partial y(k, \boldsymbol{q})}{\partial q_1} \\ \dfrac{\partial y(k, \boldsymbol{q})}{\partial q_2} \\ \cdots \\ \dfrac{\partial y(k, \boldsymbol{q})}{\partial q_r} \end{bmatrix} = \begin{bmatrix} u_k^2 \\ u_k \\ 1 \end{bmatrix}$$

$$\boldsymbol{S} = \begin{bmatrix} u_0^2 & u_1^2 & \dots & u_{K-1}^2 \\ u_0 & u_1 & \dots & u_{K-1} \\ 1 & 1 & \dots & 1 \end{bmatrix}$$

```
u = collect(-1:0.1:1)                       # nezavisno promenljiva - ulaz
K = length(u)
ni = randn(K)
y = [1.5*x^2-0.2*x+2.3 for x in u] # "izmišljeni" izlaz kao parabola
yšum = y .+ 0.1 * ni;                       # dodat šum
```

```
function parabolaTest(q, u)
    a, b, c = q
    fy = a*u.^2 .+ b*u .+ c
    fS = [u.^2; u; ones(length(u),1)]
    return fy, fS
end
q0 = [0.0, 0.0, 0.0]      # inicijalna procena vrednosti parametara
q, J, sd, n = identgrad( parabolaTest, yšum, u, q0, 0.01, kmax=1000);
```

Primer 1

poređenja rezultata

idenfitikacija metodom najmanjih kvadrata, jer je model linearan po parametrima

```
S = [u.^2 u ones(length(u),1)];
qL = S \ yšum
```

```
  1.4711262005809076
 -0.12884823900200942
  2.298520101069492
```

```
julia> q,J,sd,n = identgrad( parabolaTest,yšum,u,q0, 0.01);
0 87.1497 [ 0.0000 0.0000 0.0000 ]
1 50.3402 [ 0.2515 -0.0099 0.5960 ]
2 29.1358 [ 0.4444 -0.0191 1.0474 ]
3 16.9188 [ 0.5928 -0.0275 1.3892 ]
...
467 0.0927 [ 1.4711 -0.1288 2.2985 ]

julia> @show(q,J,sd,n);
q = [1.4710797906003539, -0.1288482390020095, 2.298538863569084]
J = 0.09274054572332398
sd = 0.09398102428698496
n = 467
```

```
julia> q,J,sd,n = identgn( parabolaTest, yšum, u, q0);
0 87.1497 [ 0.0000 0.0000 0.0000 ]
1 0.0927 [ 1.4711 -0.1288 2.2985 ]
2 0.0927 [ 1.4711 -0.1288 2.2985 ]

julia> @show(q,J,sd,n);
q = [1.4711262005809076, -0.12884823900200962, 2.2985201010694922]
J = 0.09274054317693818
sd = 0.09398102299676207
n = 2
```

Primer 2

Identifikovati parametre a, b modela $y = ae^{bx}$.

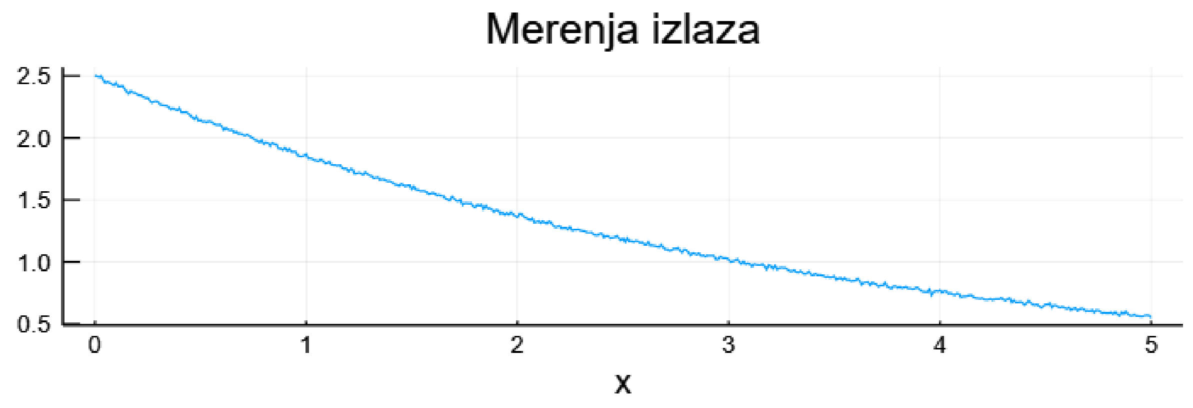
Rešenje: f-je osetljivosti su:

$$\frac{\partial y}{\partial a} = e^{bx}, \qquad \frac{\partial y}{\partial b} = axe^{bx}$$

```
function zad6_25(q,x)
   a, b = q
   # model
   fy = a*exp.(b*x)
   # f-je osetljivosti
   fo = [exp.(b*x)
         a*x .* exp.(b*x)]
   return fy, fo
end
```

```
x = collect(0:0.01:5)          # ulaz
K = length(x)                  # broj merenja
ni = randn(K)                  # šum
qtacno = [2.5; -0.3]           # parametari
y, _ = zad6_25(qtacno, x)
yšum = y + 0.01 * ni           # izlaz
```

```
julia> h=0.0001;          # h
julia> q0 = [1.0, -0.1];  # početna procena parametara
julia> q1,J,sd,n,_ = identgrad(zad6_25,yšum,x,q0,h,kmax=10000);
julia> @show(q1,J,sd,n);
q1 = [2.497138208329451, -0.2994743427264777]
J = 0.02820610362786854
sd = 0.01061024443061481
n = 1369

julia> q2, J, sd, n, _ = identgn(zad6_25, yšum, x, q0);
julia> @show(q2,J,sd,n);
q2 = [2.497289158475927, -0.2994999770072054]
J = 0.02819844508290627
sd = 0.010610100356453489
n = 6
```



Primer 3

Identifikovati parametre a, b, c, d modela $y = ae^{bx}\sin(cx + d)$.

Rešenje: f-je osetljivosti

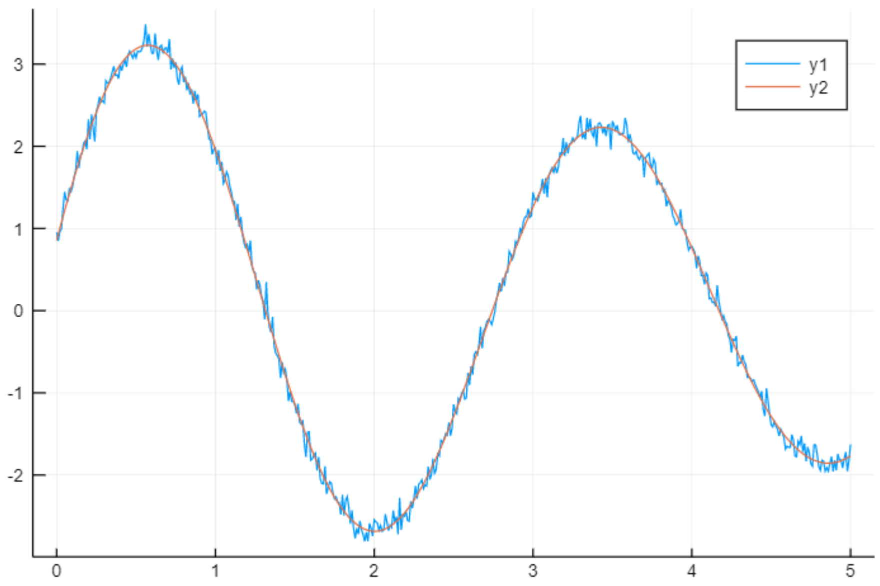
$$\frac{\partial y}{\partial a} = e^{bx}\sin(cx + d)$$

$$\frac{\partial y}{\partial b} = axe^{bx}\sin(cx + d)$$

$$\frac{\partial y}{\partial c} = axe^{bx}\cos(cx + d)$$

$$\frac{\partial y}{\partial d} = ae^{bx}\cos(cx + d)$$

```
function zad6_28(q,x)
    a, b, c, d = q
    ye = exp.(b*x)
    ys = sin.(c*x .+ d)
    yc = cos.(c*x .+ d)

    fy = a * ye .* ys
    fo = [ ye.*ys  a*ye.*ys.*x  a*ye.*yc.*x  a*ye.*yc ]'
    return fy, fo
end;
```



… primer 3

```
julia> q0 = [0.01, -0.02, -0.03, π];
julia> q1, J1, sd, n, _ = identmarq(zad6_28, yšum, u, q0, h=0.0001, kmax=300)
([-3.4839337355432742, -0.129344781188888, -2.199722938821382, 6.034713164105931],
2.3508511951341258, 0.09687432843389145, 44, Any[])

julia> q0 = [3.1, 0.01, 0.2, 0.02];
julia> q2, J2, sd, n, _ = identmarq(zad6_28, yšum, u, q0, h=0.0001, kmax=300)
([3.483933754361463, -0.12934478301516716, -2.19972293948369, 2.893120512237815],
2.350851195128016, 0.09687432843376546, 40, Any[])

julia> q0 = [-3.1, 0.01, 0.0, 0.0];
julia> q3, J3, sd, n, _ = identmarq(zad6_28, yšum, u, q0, h=0.0001, kmax=300)
([-3.4839337510626933, -0.12934478269369945, 2.199722939367904, -2.8931205119368513],
2.3508511951283397, 0.09687432843377224, 40, Any[])

julia> [q1 q2 q3]
4×3 Array{Float64,2}:
 -3.48393    3.48393   -3.48393
 -0.129345  -0.129345  -0.129345
 -2.19972   -2.19972    2.19972
  6.03471    2.89312   -2.89312
```

Može se uočiti da su sva tri računanja uspešno sprovedena, i da se identifikovane vrednosti parametara odnose na isti model. Periodičnost i neparnost funkcije *sin* je omogućila da ne postoji jedinstveno rešenje za tražene parametre.